

Universidad Nacional de Colombia

Introducción a la Inteligencia Artificial

Trabajo Práctico 1

Akinator de Super heroes

Integrantes:
Marcela Botero Bedoya
Santiago Barrera Tóvar
Daniel Felipe Villegas Ibarra
Sebastián Vargas García

Profesor:
Jaime Alberto Guzmán Luna

Sede Medellín, Colombia

29 de septiembre de 2026

AdivinaTron: sistema inteligente híbrido para la identificación de personajes de superhéroes

Abstract. Este proyecto presenta el diseño e implementación de un sistema inteligente interactivo para la inferencia y clasificación de personajes (héroes y villanos) dentro de un universo delimitado. La dinámica sigue un esquema de deducción interactiva: el usuario piensa en un personaje del catálogo inicial y el sistema ejecuta una secuencia adaptativa de preguntas de descarte que reduce progresivamente el espacio de búsqueda hasta identificar al personaje elegido.

La solución integra tres componentes. Un módulo de Ontologías modela conceptualmente los personajes, sus atributos y sus categorías jerárquicas, lo que permite deducir de forma automática y semántica conocimiento implícito a partir de los hechos declarados explícitamente. Un módulo de Lógica Difusa gestiona la incertidumbre y resuelve situaciones de ambigüedad mediante la evaluación de variables continuas como los niveles de poder, amenaza y popularidad. Finalmente, un módulo de Sistema Experto integra el conocimiento estructurado por la ontología y ejecuta un motor de reglas lógicas que gestiona el flujo de preguntas y el filtrado de candidatos; ante escenarios de empate, recurre a la lógica difusa en los puntos críticos para afinar la decisión y garantizar la identificación precisa del personaje.

1. Explicación del Dominio: Sistema Deductivo de Personajes (AdivinaTron)

El ejercicio consiste en la réplica a baja escala del juego interactivo *Akinator*, cuyo objetivo es inferir el personaje en el que piensa el jugador mediante una secuencia de preguntas y respuestas. En esta implementación, denominada *AdivinaTron*, el espacio de búsqueda se restringe a un conjunto reducido de personajes de los universos de Marvel y DC, clasificados en héroes y villanos.

Al inicio de la sesión el sistema presenta todos los personajes disponibles. A continuación, mediante un esquema de descarte basado en preguntas con respuesta Sí/No, AdivinaTron filtra el conjunto de hechos activos hasta aislar al personaje objetivo.

Cuando el filtrado lógico no converge hacia un único candidato y se genera un empate entre dos personajes, se activa un subsistema de clasificación basado en lógica difusa. Este módulo evalúa variables continuas (poder, amenaza y popularidad) para determinar cuantitativamente cuál de las entidades se aproxima mejor al perfil hipotetizado por el usuario.

2. Módulo de Ontología (RDFLib y OWL-RL)

La finalidad de este módulo es almacenar de forma estructurada el conocimiento sobre los personajes (héroes y villanos) y sus propiedades en un grafo RDF. Esta información constituye la base de conocimiento que alimenta al módulo de sistema experto, de modo que este pueda razonar sobre ella hasta llegar al personaje elegido. La ontología se implementó en Python con la librería **RDFLib**, el razonamiento se realizó con **OWL-RL** (DeductiveClosure con RDFS_Semantics) y la serialización final se hizo en el lenguaje **Turtle**.

En adelante, las referencias a líneas corresponden al módulo de Ontología en el Google Colab.

2.1. Inicialización del grafo y espacios de nombres

El módulo comienza importando las clases de rdflib necesarias (Graph, Namespace, URIRef, Literal), los vocabularios predefinidos (RDF, RDFS, XSD, FOAF, DCTERMS) y, desde owlrl, el razonador (DeductiveClosure, RDFS_Semantics). Posteriormente se crea el grafo vacío "g" y el espacio de nombres propio de la ontología, EX, con la URI base "http://ejemplo.org/superheroes/". Con g.bind(...) se asocia un prefijo legible a cada espacio de nombres, de manera que la salida en Turtle use nombres cortos (ex:Batman, foaf:name) en lugar de URIs completas.

2.2. Definición de clases y jerarquías

2.2.1. Clases

Se definieron **13 clases** (rdfs:Class) con el fin de estructurar formalmente el dominio de los personajes de ambos universos y sus atributos asociados. Las clases se agrupan en tres ejes: la **clasificación** del personaje (Personaje, Superheroe, Supervillano, HeroeMarvel, HeroeDC, VillanoMarvel, VillanoDC), su **origen** (Humano, HumanoTecnologico, HumanoMutado, Alienigena) y sus **atributos** principales (Superpoder, Equipamiento).

Para simplificar la construcción del grafo, las clases se almacenan en una lista de tuplas (URI, etiqueta, descripción) (línea 20) y un ciclo for (líneas 37–40) las recorre insertando, para cada una, tres tripletas: su tipo rdf:type rdfs:Class, su rdfs:label y su dcterms:description. Esto garantiza un tratamiento uniforme y evita repetir código.

```
clases = [
     (EX.Personaje, "Personaje", "Clase raíz para entidades del universo de cómics."),
    (EX.Superheroe, "Superhéroe", "Personaje enfocado en proteger la humanidad."),
     ...
]
for uri_c, label, desc in clases:
    g.add((uri_c, RDF.type, RDFS.Class))
    g.add((uri_c, RDFS.label, Literal(label)))
    g.add((uri_c, DCTERMS.description, Literal(desc)))
```

Tabla 1. Clases de la ontología, con su etiqueta y descripción (líneas 20–34)

Clase	rdfs:label	dcterms:description
ex:Personaje	Personaje	Clase raíz para entidades del universo de cómics.
ex:Superheroe	Superhéroe	Personaje enfocado en proteger la humanidad.
ex:Supervillano	Supervillano	Personaje antagonista que genera caos.
ex:HeroeMarvel	Héroe de Marvel	Superhéroe de la editorial Marvel Comics.
ex:HeroeDC	Héroe de DC	Superhéroe de la editorial DC Comics.
ex:VillanoMarvel	Villano de Marvel	Supervillano de la editorial Marvel Comics.
ex:VillanoDC	Villano de DC	Supervillano de la editorial DC Comics.
ex:Humano	Humano	Personaje de origen biológico terrestre.
ex:HumanoTecnologico	Humano Tecnológico	Humano que utiliza gadgets o tecnología sin poderes nativos.
ex:HumanoMutado	Humano Mutado	Humano con alteración biológica o genética.

ex:Alienigena	Alienígena	Personaje no humano proveniente del espacio exterior o reinos extra-dimensionales.
ex:Superpoder	Superpoder	Habilidad sobrehumana o metahumana.
ex:Equipamiento	Equipamiento	Artefacto, arma o tecnología empleada.

2.2.2. Relaciones de jerarquía (subclases): uso de rdfs:subClassOf

Se declararon **10 relaciones** rdfs:subClassOf (líneas 44–53), que forman una jerarquía de hasta tres niveles cuya raíz es Personaje. Las clases Superpoder y Equipamiento no participan en la jerarquía porque describen atributos y no tipos de personaje.

Tabla 2. Relaciones rdfs:subClassOf

Subclase	Relación	Superclase
ex:Superheroe	rdfs:subClassOf	ex:Personaje
ex:Supervillano	rdfs:subClassOf	ex:Personaje
ex:HeroeMarvel	rdfs:subClassOf	ex:Superheroe
ex:HeroeDC	rdfs:subClassOf	ex:Superheroe
ex:VillanoMarvel	rdfs:subClassOf	ex:Supervillano
ex:VillanoDC	rdfs:subClassOf	ex:Supervillano
ex:Humano	rdfs:subClassOf	ex:Personaje
ex:HumanoTecnologico	rdfs:subClassOf	ex:Humano
ex:HumanoMutado	rdfs:subClassOf	ex:Humano
ex:Alienigena	rdfs:subClassOf	ex:Personaje

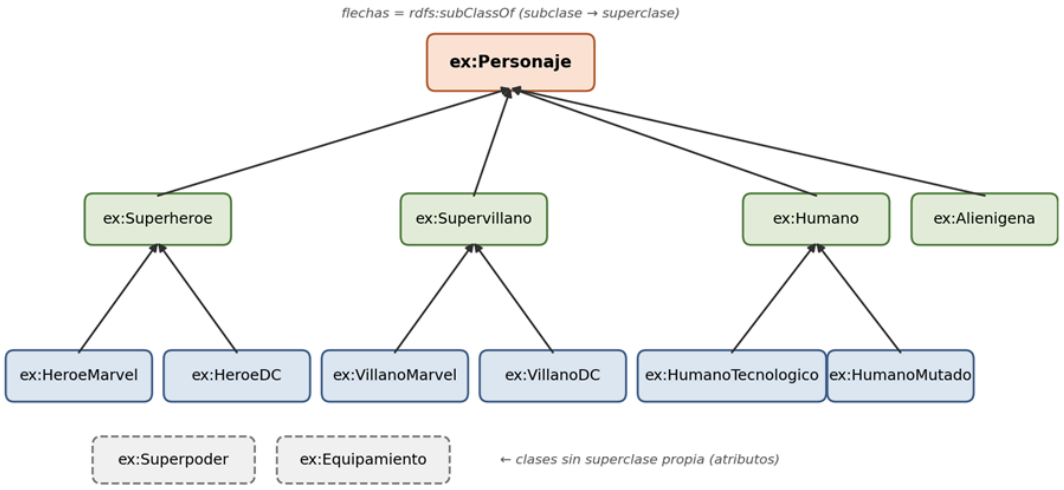


Fig. 1: Jerarquía de clases de la ontología (rdfs:subClassOf)

Cada personaje se modela con **dos tipos independientes**: uno de *rol y editorial* (por ejemplo HeroeDC) y otro de *origen* (por ejemplo Alienigena). Esta estructura permite que el razonador deduzca automáticamente nuevas pertenencias; por ejemplo, si un personaje es HumanoMutado, se infiere que también es Humano y, por transitividad, Personaje. Estas deducciones son la base para que el sistema experto formule preguntas como «¿tu personaje es humano?» o «¿es un héroe?» sin que dichas categorías hayan sido declaradas una por una.

2.3. Definición de propiedades, dominios, rangos y jerarquía de propiedades

2.3.1. Propiedades

Se declararon **11 propiedades RDF** (rdf:Property): 9 definidas en una lista de tuplas (propiedad, dominio, rango, etiqueta, descripción) (líneas 57–67) y 2 subpropiedades (líneas 76–85). De ellas, 10 son propias del dominio (prefijo ex:) y 1 es reutilizada del vocabulario FOAF (foaf:name). Un ciclo (líneas 69–74) agrega al grafo, para cada propiedad de la lista, su tipo, su rdfs:domain, su rdfs:range, su rdfs:label y su dcterms:description.

```
for propiedad_uri, domain_uri, range_uri, label, desc in props:
    g.add((propiedad_uri, RDF.type, RDF.Property))
    g.add((propiedad_uri, RDFS.domain, domain_uri))
    g.add((propiedad_uri, RDFS.range, range_uri))
    g.add((propiedad_uri, RDFS.label, Literal(label)))
    g.add((propiedad_uri, DCTERMS.description, Literal(desc)))
```

Tabla 3. Propiedades con dominio y rango

Propiedad	Dominio	Rango	rdfs:label	dcterms:description
foaf:name	ex:Personaje	xsd:string	Nombre Real	Nombre civil o de identidad real (Vocabulario FOAF).
ex:tienePoder	ex:Personaje	ex:Superpoder	Tiene Poder	Relaciona un personaje con su habilidad innata.
ex:poseeArtefacto	ex:Personaje	ex:Equipamiento	Posee Artefacto	Relaciona un personaje con su equipamiento.
ex:esEnemigoDe	ex:Personaje	ex:Personaje	Es Enemigo De	Rivalidad combativa entre dos personajes.
ex:usaIdentidadOculta	ex:Personaje	xsd:boolean	Usa Identidad Oculta	Indica si oculta su cara/identidad.
ex:valorPopularidad	ex:Personaje	xsd:integer	Valor Popularidad	Métrica de 0 a 100 para lógica difusa.
ex:valorAmenaza	ex:Personaje	xsd:integer	Valor Amenaza	Métrica de 0 a 10 para lógica difusa.
ex:valorPoder	ex:Personaje	xsd:integer	Valor Poder	Métrica de 0 a 100 para lógica difusa.
ex:tieneCapa	ex:Personaje	xsd:boolean	Tiene Capa	Indica si el personaje viste capa en su traje.

Las tres propiedades numéricas (valorPopularidad, valorAmenaza y valorPoder) son el puente con el módulo de lógica difusa: proveen las variables continuas que ese módulo fuzzifica para desempatar

candidatos. Los intervalos válidos (0–100 y 0–10) se documentan en la descripción, ya que RDFS no permite restringir un rango numérico.

2.3.2. Jerarquía de propiedades: uso de rdfs:subPropertyOf

Se definieron **2 subpropiedades** que especializan propiedades más generales (líneas 76–90). Estas se declaran sin rdfs:domain ni rdfs:range propios, pues los heredan de su superpropiedad: cuando el razonador propaga una tripleta hacia la propiedad general, se le aplican el dominio y el rango de esta.

Tabla 4. Subpropiedades

Subpropiedad	Relación	Superpropiedad	dcterms:description
`ex:poseeArmaMitica`	rdfs:subPropertyOf	`ex:poseeArtefacto`	Subpropiedad para armas divinas/míticas.
`ex:esArchienemigoDe`	rdfs:subPropertyOf	`ex:esEnemigoDe`	Subpropiedad para archirrivalidad histórica.

Así, afirmar que Thor *posee un arma mítica* (Mjolnir) implica que Thor *posee un artefacto*; y afirmar que Batman es *archienemigo* del Joker implica que son *enemigos*. La relación inversa no se cumple: no todo enemigo es un archienemigo. Las rivalidades de archienemigos (Batman–Joker, Flash–Reverse Flash y Superman–Lex Luthor) se declararon en ambos sentidos, ya que RDFS no ofrece propiedades simétricas.

Implementación en el código: propiedades y subpropiedades entre las líneas 55 y 90.

2.4. Uso de rangos: clases (URIs) y literales tipados

En la definición de las propiedades se emplearon dos tipos de rango:

- **Rangos como clases (URIs):** Relacionan individuos entre sí. Ejemplos: ex:tienePoder (rango ex:Superpoder), ex:poseeArtefacto (rango ex:Equipamiento) y ex:esEnemigoDe (rango ex:Personaje).
- **Rangos como literales tipados (XML Schema):** Asignan valores concretos con un tipo de dato explícito. Ejemplos: ex:usaIdentidadOculta y ex:tieneCapa (xsd:boolean), y ex:valorPoder (xsd:integer).

Los literales se crean siempre indicando su tipo mediante el parámetro datatype, por ejemplo:

```
g.add((EX.Wolverine, EX.usaIdentidadOculta, Literal(True, datatype=XSD.boolean)))
g.add((EX.Wolverine, EX.valorPopularidad, Literal(80, datatype=XSD.integer)))
```

Esto combina la información estructural (relaciones entre clases) con datos específicos (valores literales), vinculando la estructura conceptual del universo de cómics con variables numéricas y lógicas que aumentan la precisión del razonamiento.

2.5. Uso de vocabularios estándar (FOAF y Dublin Core)

FOAF. Se reutilizó la propiedad foaf:name (línea 58) para almacenar el **nombre real** de cada personaje, diferenciándolo de su alias de héroe o villano. El alias se representa mediante el identificador del individuo (por ejemplo ex:Batman) y la identidad civil mediante foaf:name. La propiedad recibe valores de tipo xsd:string y se asigna al instanciar cada personaje.

Tabla 5. Ejemplos de alias y nombre real (foaf:name)

Alias (individuo)	foaf:name	Observación
ex:Batman	Bruce Wayne	Identidad civil del héroe
ex:Superman	Clark Kent	Nombre terrestre del kryptoniano
ex:Wolverine	Logan	Nombre real del héroe mutante
ex:SpiderMan	Peter Parker	Identidad civil del héroe
ex:Joker	Desconocido	Su verdadero nombre no es conocido en el canon

Dublin Core (DCTERMS). Se empleó dcterms:description para asociar una explicación conceptual a cada clase (línea 40) y a cada propiedad y subpropiedad (líneas 74 y 85). Mientras que rdfs:label aporta el nombre legible del elemento, dcterms:description documenta su significado dentro del dominio, de modo que la ontología sea autoexplicativa para quien la consulte.

2.6. Uso de rdfs:label

Se asignó una etiqueta rdfs:label a **todas las clases y todas las propiedades** de la ontología, con el fin de describir de forma legible cada elemento. En el código, las etiquetas se agregan de forma automatizada dentro de los ciclos de las líneas 39 (clases), 73 (propiedades) y 84 (subpropiedades).

```
g.add((uri_c, RDFS.label, Literal(label)))              # línea 39
g.add((propiedad_uri, RDFS.label, Literal(label)))      # línea 73
g.add((subprop_uri, RDFS.label, Literal(label)))        # línea 84
```

2.7. Instanciación de las clases

Se instanciaron los individuos de todas las clases del modelo. Los poderes, el equipamiento y los personajes se agregaron de la siguiente manera:

- **Superpoder (13 individuos, líneas 94–106):** Sentido Arácnido, Fuerza Mejorada, Control del Trueno, Poder Cósmico, Súper Velocidad, Vuelo, Fuerza Sobrehumana, Fuerza Divina, Simbionte, Rayos Omega, Comunicación Marina, Hechicería y Factor Curativo.
- **Equipamiento (14 individuos explícitos, líneas 111–115):** Armadura Tech, Arco Gadgets, Escudo de Vibranium, Mjolnir, Tabla Cósmica, Batarang, Planeador de Bombas, Traje de Alas Tech, Toxina de la Risa, Acertijos, Traje de Kryptonita, Armadura Misticotech, Tridente de Neptuno y Garras de Adamantium. Los dos individuos restantes, Guantelete y LazoDeLaVerdad, no se declaran en esta lista: los deduce el razonador a partir del rango de poseeArtefacto (ver sección 2.10.2), para un total de 16.
- **Personajes (26 individuos, líneas 119–408):** 7 héroes de Marvel, 7 héroes de DC, 6 villanos de Marvel y 6 villanos de DC.

Por cada personaje se declaran: (i) su tipo de editorial/rol y su tipo de origen; (ii) su nombre real con foaf:name; (iii) sus poderes con ex:tienePoder y su equipamiento con ex:poseeArtefacto o ex:poseeArmaMitica, según corresponda; (iv) si usa identidad oculta y, cuando aplica, si tiene capa; (v) las tres métricas numéricas para la lógica difusa; y (vi) en los casos de rivalidad histórica, su archienemigo. Como ejemplo, así se instancia a Batman (líneas 210–219):

```
g.add((EX.Batman, RDF.type, EX.HeroeDC))
g.add((EX.Batman, RDF.type, EX.HumanoTecnologico))
```

```
g.add((EX.Batman, EX.tieneCapa, Literal(True, datatype=XSD.boolean)))
g.add((EX.Batman, EX.esArchienemigoDe, EX.Joker))
g.add((EX.Batman, FOAF.name, Literal("Bruce Wayne", datatype=XSD.string)))
g.add((EX.Batman, EX.poseeArtefacto, EX.Batarang))
g.add((EX.Batman, EX.usaIdentidadOculta, Literal(True, datatype=XSD.boolean)))
g.add((EX.Batman, EX.valorPopularidad, Literal(99, datatype=XSD.integer)))
g.add((EX.Batman, EX.valorAmenaza, Literal(5, datatype=XSD.integer)))
g.add((EX.Batman, EX.valorPoder, Literal(28, datatype=XSD.integer)))
```

Tabla 6. Individuos por clase (declarados explícitamente y totales tras el razonamiento)

Clase	Explícitos	Tras razonar
ex:Personaje	0	26
ex:Superheroe	0	14
ex:Supervillano	0	12
ex:HeroeMarvel	7	7
ex:HeroeDC	7	7
ex:VillanoMarvel	6	6
ex:VillanoDC	6	6
ex:Humano	0	19
ex:HumanoTecnologico	10	10
ex:HumanoMutado	9	9
ex:Alienigena	7	7
ex:Superpoder	13	13
ex:Equipamiento	14	16

Las clases generales (Personaje, Superheroe, Supervillano y Humano) **no tienen individuos declarados de forma explícita**: cada personaje se tipifica únicamente con su clase más específica y el razonador deduce el resto a partir de la jerarquía. Esto evita la redundancia y demuestra el valor de la inferencia. En Equipamiento, dos de sus 16 individuos (Guantelete y LazoDeLaVerdad) tampoco se declaran, sino que se deducen.

2.8 Estado del grafo antes del razonamiento

Antes de activar el motor de inferencia, el script registra el número de tripletas explícitas (tripletas_antes = len(g), línea 410) y comprueba, mediante el operador in, que los hechos que se deducirán todavía **no** existen en el grafo (líneas 413–431). El grafo contiene **363 tripletas explícitas**, distribuidas así:

Los seis casos verificados devuelven False en esta etapa: Thor no es aún Superheroe; Superman no es aún Personaje; Wonder Woman no es aún Personaje ni LazoDeLaVerdad es aún Equipamiento; y las tripletas Thor poseeArtefacto Mjolnir y Joker esEnemigoDe Batman todavía no existen.

2.9. Razonamiento con DeductiveClosure (RDFS Semantics)

El razonamiento se realizó sobre la ontología final del dominio con la librería owlrl, en la línea 436.

```
DeductiveClosure(RDFS_Semantics).expand(g)
```

2.9.1. Comparación del grafo antes y después

Tras el razonamiento, el script vuelve a medir el grafo (tripletas_despues = len(g), línea 437) e imprime la diferencia como número de nuevas afirmaciones inferidas (líneas 439–441).

2.10. Hechos generados por inferencia RDFS

2.10.1. Jerarquía de clases (rdfs:subClassOf)

Los siguientes hechos se infirieron a partir de las relaciones de subclase declaradas en la ontología (Caso 1, líneas 446–451):

Caso 1.1 — HeroeMarvel ⊑ Superheroe. Al declarar:

```
g.add((EX.Thor, RDF.type, EX.HeroeMarvel))
```

se infiere: ex:Thor rdf:type ex:Superheroe. Antes del razonamiento la comprobación devolvía False; después devuelve True.

Caso 1.2 — HeroeDC ⊑ Superheroe ⊑ Personaje. Al declarar:

```
g.add((EX.Superman, RDF.type, EX.HeroeDC))
```

se infiere: ex:Superman rdf:type ex:Personaje. Aquí la deducción atraviesa dos niveles de la jerarquía gracias a la transitividad de rdfs:subClassOf (HeroeDC ⊑ Personaje). Superman, además, es Alienigena, que también es subclase de Personaje, por lo que el hecho se deduce por dos caminos.

2.10.2 Dominio y rango de propiedades (rdfs:domain y rdfs:range)

Este caso documenta la pertenencia de individuos a una clase deducida desde el dominio y el rango de una propiedad (Caso 2, líneas 453–459). Wonder Woman se relaciona con su lazo mediante:

```
g.add((EX.WonderWoman, EX.poseeArmaMitica, EX.LazoDeLaVerdad))    # línea 259
```

El individuo ex:LazoDeLaVerdad **no se declaró** en la lista de equipamientos (líneas 111–113) ni tiene ningún rdf:type explícito. Aun así, el razonador lo clasifica en tres pasos, ilustrados en la Fig. 2:

- Como poseeArmaMitica ⊑ poseeArtefacto, se infiere ex:WonderWoman ex:poseeArtefacto ex:LazoDeLaVerdad.
- **Dominio:** el dominio de poseeArtefacto es Personaje, luego ex:WonderWoman rdf:type ex:Personaje.
- **Rango:** el rango de poseeArtefacto es Equipamiento, luego ex:LazoDeLaVerdad rdf:type ex:Equipamiento.

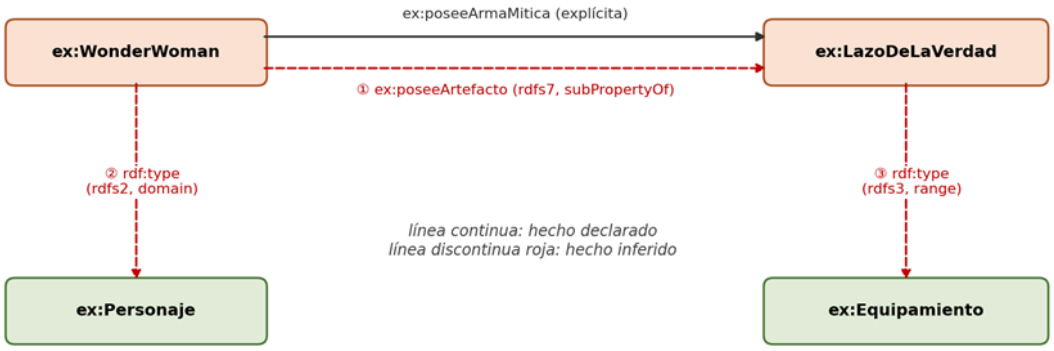


Fig. 2: Deducción de la pertenencia a clases desde el dominio y el rango de poseeArtefacto

Antes del razonamiento, (ex:LazoDeLaVerdad, rdf:type, ex:Equipamiento) in g devuelve False; después, True.

2.10.3 Subpropiedades (rdfs:subPropertyOf)

En la ontología se definió que poseeArmaMitica es subpropiedad de poseeArtefacto y que esArchienemigoDe es subpropiedad de esEnemigoDe (líneas 89–90):

```
g.add((EX.poseeArmaMitica, RDFS.subPropertyOf, EX.poseeArtefacto))
g.add((EX.esArchienemigoDe, RDFS.subPropertyOf, EX.esEnemigoDe))
```

Esto significa que cualquier tripleta que use la propiedad específica genera automáticamente una nueva tripleta con la propiedad general (Caso 3, líneas 461–466). Así, al declarar Thor poseeArmaMitica Mjolnir se infiere Thor poseeArtefacto Mjolnir, y al declarar Joker esArchienemigoDe Batman se infiere Joker esEnemigoDe Batman. En total, el razonador produce 11 tripletas nuevas por esta vía:

Tabla 7. Tripletas inferidas por subpropiedad

Sujeto	Propiedad inferida	Objeto
ex:Thor	ex:poseeArtefacto	ex:Mjolnir
ex:SilverSurfer	ex:poseeArtefacto	ex:TablaCosmica
ex:Thanos	ex:poseeArtefacto	ex:Guantelete
ex:WonderWoman	ex:poseeArtefacto	ex:LazoDeLaVerdad
ex:Aquaman	ex:poseeArtefacto	ex:TridenteDeNeptuno
ex:Batman	ex:esEnemigoDe	ex:Joker
ex:Joker	ex:esEnemigoDe	ex:Batman
ex:Flash	ex:esEnemigoDe	ex:ReverseFlash
ex:ReverseFlash	ex:esEnemigoDe	ex:Flash
ex:Superman	ex:esEnemigoDe	ex:LexLuthor
ex:LexLuthor	ex:esEnemigoDe	ex:Superman

2.11. Cuatro hechos inferidos automáticamente

Adicionalmente a los casos anteriores, se evidencian cuatro hechos concretos que no fueron declarados y que son producto exclusivo del razonador: todos ellos provienen de la lista hechos_ejemplo que imprime el script (líneas 468–479):

Tabla 8. Hechos inferidos con su justificación

N.º	Hecho inferido	Justificación
1	`ex:Superman rdf:type ex:Personaje`	Subclase: `Alienigena` ⊑ `Personaje` (y `HeroeDC` ⊑ `Superheroe` ⊑ `Personaje`)
2	`ex:Thanos rdf:type ex:Personaje`	Subclase: `VillanoMarvel` ⊑ `Supervillano` ⊑ `Personaje`
3	`ex:IronMan rdf:type ex:Humano`	Subclase: `HumanoTecnologico` ⊑ `Humano`
4	`ex:Guantelete rdf:type ex:Equipamiento`	Subpropiedad + rango: `poseeArmaMitica` ⊑ `poseeArtefacto`, rango `Equipamiento`

2.12. Serialización en Turtle

Como paso final (líneas 482–485), el grafo se serializa en el formato **Turtle** con g.serialize(destination="ontologia_generada.ttl", format="turtle"). Este archivo .ttl es guardado en la sección de archivos del Google Colab

3. Módulo de Componente de Lógica difusa

3.1. Rol del módulo dentro del sistema híbrido

El sistema desarrollado funciona como un Akinator de personajes del universo de cómics (Marvel y DC). El sistema experto formula preguntas de sí/no cuyas respuestas descartan personajes de la base de conocimiento, la cual proviene de la ontología. Sin embargo, al finalizar la ronda de preguntas es posible que queden dos candidatos que no se pueden distinguir con respuestas booleanas, pues la información restante es de naturaleza gradual: qué tan poderoso, qué tan amenazante o qué tan popular es un personaje no admite una respuesta de sí o no. Este es el problema para el que se incorpora la lógica difusa.

El módulo difuso cumple entonces la función de desempate. El flujo es el siguiente:

- **Cuando quedan exactamente dos personajes** el sistema experto pasa a la fase "desempate" y solicita al usuario tres valores que describan al personaje que piensa: poder (0–100), amenaza (0–10) y popularidad (0–100).
- **Se calcula el perfil de impacto del usuario:** los tres valores se ingresan al sistema de inferencia difusa y se defuzzifican mediante el centroide, obteniendo un único valor en [0, 100] que se almacena como el hecho `PerfilDifuso(impacto_esperado=...)`.
- **Se calcula el perfil de impacto de cada candidato:** para cada uno de los personajes se toman sus atributos `valorPoder`, `valorAmenaza` y `valorPopularidad` —traducidos desde la ontología como valores enteros tipados con `xsd:integer`— y se evalúa el mismo sistema difuso.
- **Decisión:** se selecciona el personaje cuyo perfil de impacto esté más cerca del valor del usuario, es decir, el que minimice limpacto_esperado – impacto_candidatol.

El resultado de la lógica difusa (hecho PerfilDifuso) es una precondición de la regla de desempate del sistema experto, y las entradas del módulo difuso provienen de las propiedades numéricas de la ontología (ex:valorPoder, ex:valorAmenaza y ex:valorPopularidad), cuyos universos de discurso se definieron alineados con los rangos declarados en la propia ontología.

Referencia al código. La implementación se encuentra en la sección "Módulo de Lógica Difusa, la invocación desde el sistema experto se realiza en las reglas decision_final y desempate.

3.2. Variables de entrada (Antecedentes)

Se definieron tres variables difusas de entrada con la clase ctrl.Antecedent de Scikit-Fuzzy. Cada una tiene su universo del discurso y sus conjuntos difusos, y corresponde a una propiedad numérica de la ontología.

1. Poder - ex:valorPoder

Universo: [0, 100]. Mide la capacidad de combate/habilidades del personaje.

Conjunto difuso	Función	Parámetros	Interpretación
Bajo	Triangular	[0, 0, 35]	Humanos con entrenamiento o gadgets
Medio	Trapezoidal	[25, 35, 65, 70]	Sobrehumanos y armas avanzadas
Alto	Gaussiana	$\mu = 100, \sigma = 12$	Nivel universal/cósmico
Muy_Alto	Modificador (concentración)	$(\mu Alto)^2$	Poder extremo
Ligeramente_Alto	Modificador (dilatación)	$\sqrt{(\mu Alto)}$	Poder algo por encima de lo medio

2. Amenaza - ex:valorAmenaza

Universo: [0, 10]. Representa la magnitud de la destrucción que causaría el personaje (si fuera o es malvado): de ciudades, del mundo o del universo.

Conjunto difuso	Función	Parámetros	Interpretación
Baja	Trapezoidal	[0, 0, 2, 3]	Amenaza a nivel de ciudad
Media	Gaussiana	$\mu = 5, \sigma = 1$	Amenaza a nivel mundial
Alta	Triangular	[7, 10, 10]	Amenaza a nivel universal
Muy_Alta	Modificador (concentración)	$(\mu Alta)^2$	Amenaza universal extrema
Ligeramente_Alta	Modificador (dilatación)	$\sqrt{(\mu Alta)}$	Amenaza algo elevada

3. Popularidad - ex:valorPopularidad

Universo: [0, 100. Indica qué tan conocido es el personaje para el público general.

Conjunto difuso	Función	Parámetros	Interpretación
Baja	Triangular	[0, 0, 30]	Poco popular
Media	Gaussiana	$\mu = 50, \sigma = 10$	Medianamente conocido
Alta	Trapezoidal	[70, 80, 100, 100]	Ícono, muy conocido
Muy_Alta	Modificador (concentración)	$(\mu Alta)^2$	Popularidad máxima

3.3. Variable de salida

Perfil de impacto, definida con ctrl.Consequent. Universo: [0, 100]. Sintetiza en un único número qué tan grande es el alcance del personaje considerando poder, amenaza y popularidad conjuntamente. Es el valor que se compara entre el usuario y los candidatos. Método de defuzzificación: centroide (justificado en la sección 3.7).

Conjunto difuso	Función	Parámetros	Interpretación
Callejero	Trapezoidal	[0, 0, 10, 30]	Impacto local, a escala de barrio o ciudad
Global	Triangular	[30, 50, 70]	Impacto a escala mundial
Cósmico	Gaussiana	μ = 100, σ = 7	Impacto a escala universal

3.4. Uso de modificadores lingüísticos

Se emplearon dos modificadores lingüísticos que transforman la función de pertenencia de un conjunto base para representar mayor o menor intensidad de una condición:

- **"Muy" - concentración:** eleva la pertenencia al cuadrado. Al ser $\mu \in [0,1]$, el resultado es menor o igual al original, por lo que el conjunto se vuelve más exigente: solo los valores con pertenencia alta siguen siendo "muy" altos.

$$\mu_muy\ A(x) = (\ \mu_A(x)\ )^2$$

- **"Ligeramente" - dilatación:** aplica la raíz cuadrada. El resultado es mayor o igual al original, así que el conjunto se vuelve más permisivo y abarca valores con pertenencia parcial.

$$\mu_ligeramente\ A(x) = \sqrt{(\ \mu_A(x)\ )}$$

Aplicación en el modelo: Muy_Alto y Ligeramente_Alto sobre poder Alto; Muy_Alta y Ligeramente_Alta sobre amenaza Alta; y Muy_Alta sobre popularidad Alta.

3.5. Definición de reglas difusas

Se definieron 9 reglas difusas de tipo Mamdani que relacionan las tres variables de entrada con el perfil de impacto. Se emplearon los tres operadores lógicos difusos, representados en Scikit-Fuzzy con los símbolos &, | y ~:

- **Intersección (AND, &):** se evalúa como el mínimo de las pertenencias, min(μA, μB).
- **Unión (OR, |):** se evalúa como el máximo, max(μA, μB).
- **Complemento (NOT, ~):** se evalúa como 1 − μA.

#	Regla	Operadores	Modificadores
1	Si poder es Bajo Y amenaza es Baja, entonces perfil es Callejero	AND	—
2	Si poder es Muy_Alto Y amenaza es Muy_Alta, entonces perfil es Cosmico	AND	"Muy"
3	Si popularidad es Alta Y (poder es Medio O amenaza es Media), entonces perfil es Global	AND + OR	—
4	Si popularidad NO es Alta Y poder es Bajo, entonces perfil es Callejero	NOT + AND	—
5	Si poder es Ligeramente_Alto O popularidad es Muy_Alta, entonces perfil es Global	OR	"Ligeramente", "Muy"

6	Si amenaza es Alta Y poder NO es Bajo, entonces perfil es Cosmico	AND + NOT	—
7	Si poder es Medio Y popularidad es Baja, entonces perfil es Callejero	AND	—
8	Si popularidad es Alta Y amenaza es Muy_Alta, entonces perfil es Cosmico	AND	"Muy"
9	Si amenaza NO es Alta Y poder NO es Alto Y popularidad es Media, entonces perfil es Global	NOT (×2) + AND	—

Las reglas cubren los tres perfiles de salida: las reglas 1, 4 y 7 concluyen "Callejero" (personajes de poca escala o poca fama); las reglas 3, 5 y 9 concluyen "Global" (escala intermedia o fama alta); y las reglas 2, 6 y 8 concluyen "Cosmico" (amenaza y poder de nivel universal). Se incluyen dos casos de negación múltiple (regla 9), combinaciones AND/OR anidadas (regla 3) y el uso de ambos modificadores dentro de la misma regla (regla 5).

3.6. Mecanismo de inferencia

Se utiliza el motor de inferencia tipo Mamdani, para cada evaluación se ejecutan estos pasos:

- **Fuzzificación:** cada valor de entrada nítido se convierte en grados de pertenencia a cada conjunto difuso de su variable.
- **Evaluación de antecedentes:** se combinan los grados con AND = mínimo, OR = máximo y NOT = 1 − μ, obteniendo la fuerza de activación (w) de cada regla.
- **Implicación:** el conjunto de salida de cada regla se recorta a la altura w (mínimo entre w y la función de pertenencia del consecuente).
- **Agregación:** los conjuntos recortados de todas las reglas se unen mediante el máximo punto a punto, formando un único conjunto difuso de salida.
- **Defuzzificación:** el conjunto agregado se convierte en un número mediante el centroide (sección 3.7).

3.7. Proceso de defuzzificación y su justificación

Se utilizó el método del centroide (centro de gravedad). Dado el conjunto difuso agregado con función de pertenencia sobre el universo de salida, el valor nítido es la abscisa del centro de gravedad del área bajo la curva.

Justificación de la elección

- **Naturaleza de la aplicación (comparación entre dos candidatos).** La salida del módulo no se usa para activar un actuador sino para comparar numéricamente al personaje del usuario con dos candidatos. Por ende, se requiere un valor continuo y sensible a las entradas, de modo que entradas parecidas produzcan salidas parecidas y entradas distintas produzcan salidas distintas. El centroide cumple esto porque varía suavemente al cambiar las fuerzas de activación de las reglas.
- **Considera todas las reglas activadas.** El centroide pondera el área completa del conjunto agregado, de modo que cada regla que se dispara, aunque sea parcialmente aporta al resultado.

- **Resuelve conflictos entre conclusiones opuestas.** Cuando se activan reglas que apuntan a perfiles distintos (por ejemplo, Global y Cósmico simultáneamente, como ocurre con personajes de amenaza y poder altos), el centroide produce un valor intermedio coherente (entre 50 y 64) en lugar de escoger arbitrariamente uno de los conjuntos.
- **Significado del valor obtenido.** z* resume cuantitativamente la evidencia de las tres variables: valores cercanos a 0–30 indican perfil Callejero, alrededor de 50 perfil Global y valores altos perfil Cósmico. Al ser el mismo procedimiento para el usuario y para cada candidato, la comparación es consistente.

3.8. Gráficas de funciones de pertenencia

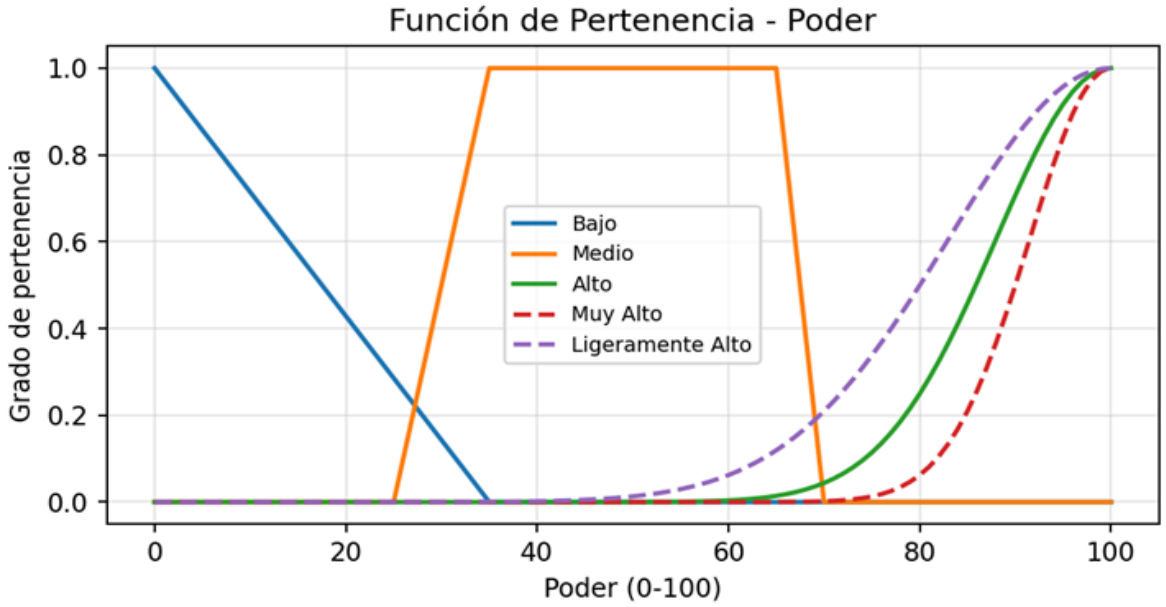


Fig. 1: Funciones de pertenencia de la variable poder (las líneas discontinuas son los modificadores).

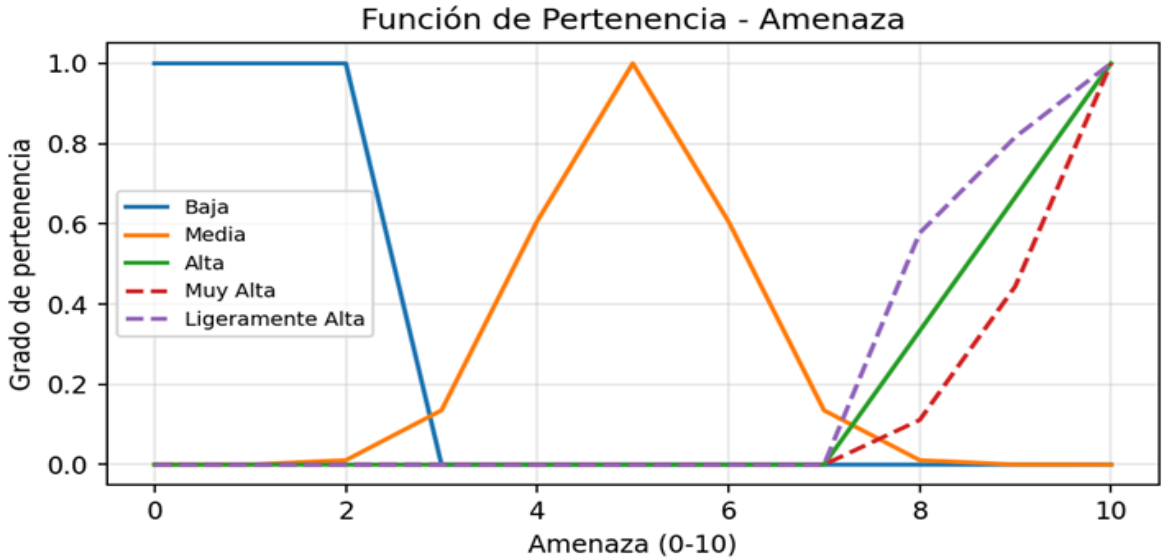


Fig. 2: Funciones de pertenencia de la variable amenaza.

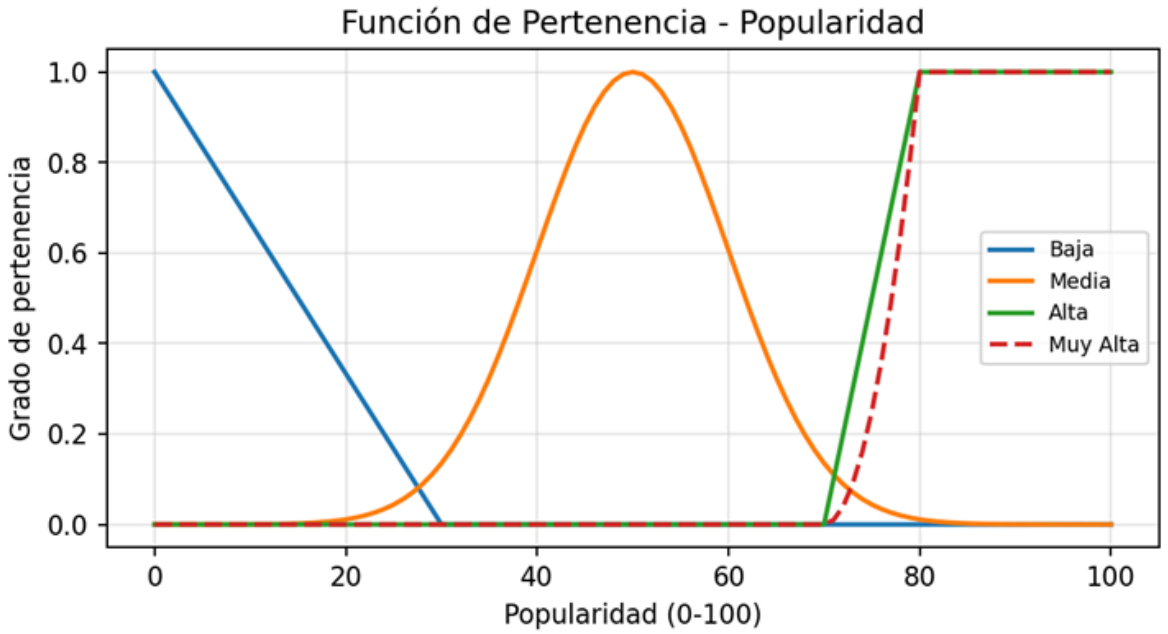


Fig. 3: Funciones de pertenencia de la variable popularidad.

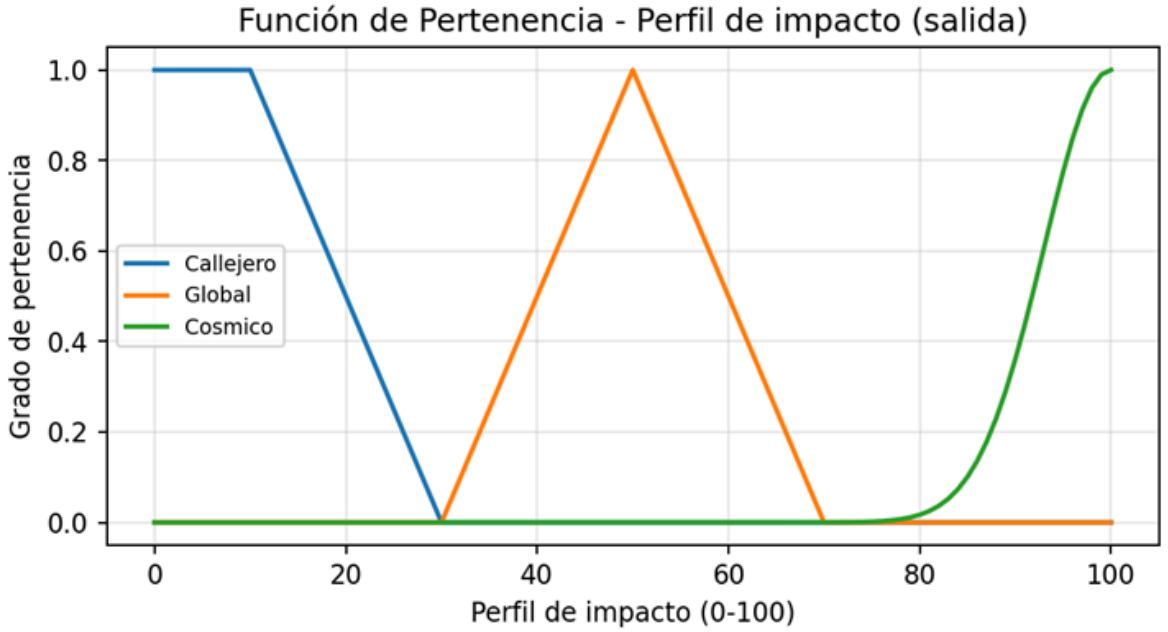


Fig. 4: Funciones de pertenencia de la variable de salida perfil_impacto.

Ejemplo del desempate

Suponga que el usuario piensa en un villano de Marvel y, tras las preguntas, quedan Thanos y Green Goblin. El usuario ingresa poder = 95, amenaza = 10 y popularidad = 85, lo que produce

impacto_esperado = 63.54. El sistema evalúa a los candidatos: Thanos = 63.54 y Green Goblin = 50.00. Las diferencias absolutas son 0.00 y 13.54, respectivamente, y por ello el sistema desempata a favor de Thanos.

4. Módulo de Sistema Experto

El módulo de sistema experto es el núcleo del juego tipo Akinator para superhéroes y supervillanos (*adivinaTron*). Está implementado con la librería **Experta** (motor de inferencia basado en reglas de producción) y se alimenta de la ontología en RDF (ontologia_generada.ttl) generada previamente con **rdflib**. El sistema formula preguntas de sí/no, descarta los personajes que no coinciden con las respuestas del usuario y, cuando quedan dos candidatos, delega la decisión final en el sistema de lógica difusa.

4.1. Definición de clases de hechos

En el sistema experto se definen siete clases de hechos que representan las entidades del dominio y el control del flujo de inferencia. Todas heredan de Fact, lo que permite su uso dentro del motor de inferencia (líneas 129 a 155).

- **Personaje**: representa a cada candidato a ser adivinado. Sus atributos se construyen a partir de la ontología e incluyen: uri (nombre local del recurso, por ejemplo SpiderMan), name (nombre real, foaf:name), un booleano por cada clase de la ontología (Superheroe, Supervillano, HeroeMarvel, HeroeDC, VillanoMarvel, VillanoDC, Humano, HumanoTecnologico, HumanoMutado, Alienigena, entre otras), un booleano por cada propiedad (tienePoder, poseeArtefacto, poseeArmaMitica, esEnemigoDe, esArchienemigoDe, tieneCapa), el booleano usaIdentidadOculta y tres valores enteros: valorPoder, valorAmenaza y valorPopularidad.
- **Respuesta**: almacena temporalmente la respuesta más reciente del usuario. Contiene el atributo preguntado (atributo) y la respuesta (valor, verdadero o falso). Es un hecho de vida corta: se declara al responder y se retracta una vez procesado.
- **AtributoEvaluado**: historial de atributos ya evaluados (nombre). Evita que el sistema repita preguntas o formule preguntas redundantes.
- **EstadoJuego**: controla la fase de inferencia. Toma tres valores en el atributo fase: "descarte", "final" y "desempate". En las dos últimas fases guarda además la lista de candidatos restantes.
- **Clase**: representa una clase de la ontología con su pregunta asociada (uri, pregunta).
- **Propiedad**: representa una propiedad de la ontología con su pregunta asociada (uri, pregunta).
- **PerfilDifuso**: almacena el impacto esperado (impacto_esperado), es decir, el valor desfuzzificado que devuelve el sistema de lógica difusa a partir del poder, la amenaza y la popularidad ingresados por el usuario. Se declara dinámicamente durante la ejecución (no forma parte de los hechos iniciales) y su presencia habilita la regla de desempate.

Además, la clase Akinator (hija de KnowledgeEngine, línea 159) agrupa todas las reglas del sistema.

4.2. Definición de reglas basadas en el conocimiento del dominio

El sistema experto implementa **34 reglas de producción** (líneas 163 a 413) que modelan el proceso de descarte de candidatos, la transición entre fases y la decisión final. Se dividen en cuatro reglas de control y 30 reglas de descarte.

Reglas de control de flujo

- **Formulación de preguntas** (descarte_preguntas): si el juego está en fase de descarte y existe un hecho Clase o Propiedad cuyo atributo aún no ha sido evaluado, se imprime la pregunta asociada, se solicita al usuario "Sí (1) o No (0)" y se declara un hecho Respuesta.
- **Procesamiento de la respuesta** (eliminar_respuesta): al existir una Respuesta, se registra el atributo como evaluado (AtributoEvaluado), se retracta la respuesta y se cuentan los candidatos que siguen vivos. Si quedan dos o menos, se retracta la fase de descarte y se declara EstadoJuego(fase="final", candidatos=[...]).
- **Decisión final** (decision_final): según el número de candidatos restantes, el sistema toma uno de tres caminos:
  - *Un candidato:* se imprime el personaje adivinado y el juego termina.
  - *Cero candidatos:* se informa que no se encontró un personaje con esos atributos.
  - *Dos candidatos:* se declara EstadoJuego(fase="desempate"), se solicita al usuario el nivel de poder (0 a 100), amenaza (0 a 10) y popularidad (0 a 100), se calcula el impacto esperado con evaluar_perfil_difuso y se declara un hecho PerfilDifuso con ese valor. Si el usuario ingresa un valor no numérico se captura el ValueError, se informa entrada inválida y el juego termina sin declarar PerfilDifuso.
- **Desempate difuso** (desempate): se activa cuando existen simultáneamente EstadoJuego(fase="desempate") y PerfilDifuso. Calcula el impacto de cada candidato con evaluar_perfil_difuso, usando sus valores valorPoder, valorAmenaza y valorPopularidad de la ontología, y lo compara con impacto_esperado. Gana el candidato con menor diferencia absoluta. Esta regla ya no interactúa con el usuario.

Reglas de descarte por atributo (30 reglas)

Para cada una de las 15 preguntas del juego existen dos reglas simétricas: una que elimina a los personajes que **sí** poseen el atributo cuando el usuario responde **No**, y otra que elimina a los que **no** lo poseen cuando responde **Sí**.

Atributo	Respuesta Sí → elimina	Respuesta No → elimina	Atributos que se marcan como ya evaluados tras un Sí
Superheroe	Superheroe=False	Superheroe=True	VillanoDC, VillanoMarvel, Supervillano
Supervillano	Supervillano=False	Supervillano=True	Superheroe, HeroeMarvel, HeroeDC
HeroeMarvel	HeroeMarvel=False	HeroeMarvel=True	Supervillano, VillanoDC, VillanoMarvel, HeroeDC, Superheroe
HeroeDC	HeroeDC=False	HeroeDC=True	Supervillano, Superheroe, HeroeMarvel, VillanoMarvel, VillanoDC
VillanoMarvel	VillanoMarvel=False	VillanoMarvel=True	Supervillano, Superheroe, HeroeMarvel, HeroeDC, VillanoDC
VillanoDC	VillanoDC=False	VillanoDC=True	Supervillano, Superheroe, HeroeMarvel, HeroeDC, VillanoMarvel
Humano	Humano=False	Humano=True	Alienigena

HumanoTecnologico	HumanoTecnologico=False	HumanoTecnologico=True	HumanoMutado, Alienigena, Humano
HumanoMutado	HumanoMutado=False	HumanoMutado=True	HumanoTecnologico, Alienigena, Humano
Alienigena	Alienigena=False	Alienigena=True	Humano, HumanoTecnologico
tienePoder	tienePoder=False	tienePoder=True	—
poseeArtefacto	poseeArtefacto=False	poseeArtefacto=True	—
esArchienemigoDe	esArchienemigoDe=False	esArchienemigoDe=True	—
usaIdentidadOculta	usaIdentidadOculta=False	usaIdentidadOculta=True	—
tieneCapa	tieneCapa=False	tieneCapa=True	—

La columna final refleja el **razonamiento sobre la jerarquía de clases** de la ontología: si el usuario confirma, por ejemplo, que su personaje es un HeroeMarvel, ya no tiene sentido preguntar si es villano, si es héroe de DC o si es superhéroe en general, por lo que esos atributos se marcan como evaluados y se omiten. Esto reduce el número de preguntas necesarias para llegar a la respuesta.

Referencia al código. Las reglas de control están en las líneas 163 a 239 y las 30 reglas de descarte en las líneas 243 a 413.

4.3. Estructura de las reglas

Las reglas siguen un patrón uniforme de *hecho disparador + hecho a evaluar*:

- **Antecedente:** una Respuesta(atributo=X, valor=V) y un Personaje cuyo atributo X tenga el valor contrario a V, junto con la fase de juego cuando es necesario (EstadoJuego(fase=...)).
- **Consecuente:** retractar (retract) al personaje que contradice la respuesta y, cuando corresponde, declarar AtributoEvaluado para las preguntas que ya quedaron resueltas por jerarquía.

Ejemplo (líneas 243 a 249): si el usuario responde que sí es un superhéroe, se retractan los personajes con Superheroe=False y se marcan como evaluados VillanoDC, VillanoMarvel y Supervillano.

Ciclo de ejecución

1. Se declara EstadoJuego(fase="descarte").
2. descarte_preguntas formula una pregunta y declara Respuesta.
3. La regla de descarte correspondiente elimina a los personajes incompatibles.
4. eliminar_respuesta registra el atributo, retracta la respuesta y cuenta candidatos.
5. Si quedan más de dos, se vuelve al paso 2. Si quedan dos o menos, se declara EstadoJuego(fase="final").
6. decision_final imprime el resultado o, con dos candidatos, declara EstadoJuego(fase="desempate"), solicita los tres valores al usuario y declara PerfilDifuso con el impacto esperado.
7. desempate compara el impacto de cada candidato con impacto_esperado y anuncia el ganador.

4.4. Niveles de prioridad

Se implementan cinco niveles de prioridad mediante el atributo salience. Los valores están escogidos para que, tras cada respuesta, primero se descarte, luego se cuente y solo entonces se pregunte de nuevo:

- **Nivel crítico (salience = 15):** decisión final (decision_final). Debe ejecutarse apenas se declara la fase final; además captura los datos del usuario y declara PerfilDifuso cuando hay dos candidatos. (Líneas 187 a 218).
- **Nivel alto (salience = 10):** las 30 reglas de descarte por atributo. Deben ejecutarse antes de contar los candidatos. (Líneas 243 a 413).
- **Nivel medio (salience = 7):** procesamiento de la respuesta y conteo de candidatos (eliminar_respuesta). (Líneas 173 a 180).
- **Nivel bajo (salience = 5):** formulación de preguntas (descarte_preguntas). Solo se pregunta cuando no queda ninguna otra regla pendiente. (Líneas 163 a 170).
- **Nivel por defecto (salience = 0):** desempate difuso (desempate). No declara prioridad explícita porque depende de los hechos EstadoJuego(fase="desempate") y PerfilDifuso, que solo existen al final del juego. (Líneas 220 a 239).

Esta jerarquización garantiza que eliminar_respuesta (7) cuente los candidatos solo después de que todas las reglas de descarte (10) hayan actuado, evitando decisiones sobre una base de hechos incompleta.

4.5. Mecanismos de control de conflictos

Para evitar conflictos entre reglas se implementan los siguientes mecanismos:

- **Prioridad (salience):** define el orden entre las reglas de la agenda, tal como se describe en la sección 4.4.
- **LIFO (Last In, First Out):** el código no configura una estrategia propia, por lo que Experta aplica su estrategia por defecto: ante igualdad de salience, se ejecutan primero las activaciones más recientes. Esto se aprovecha en las preguntas, ya que los hechos Clase y Propiedad se declaran en orden aleatorio (random.shuffle, línea 437) y por tanto el orden de las preguntas cambia en cada partida.

4.6. Control de bucles infinitos

Para evitar la ejecución repetitiva de reglas se implementan los siguientes mecanismos de tipo **No-Loop**:

- **Preguntas sin repetición:** la regla descarte_preguntas incluye la condición NOT(AtributoEvaluado(nombre=MATCH.attr)) (línea 165), de modo que un atributo ya evaluado no vuelve a preguntarse. eliminar_respuesta declara ese hecho en la línea 175.
- **Decisión final una sola vez:** decision_final incluye la condición NOT(EstadoJuego(fase="desempate")) (línea 188, marcada con el comentario # no-loop), de modo que no se dispare de nuevo una vez iniciado el desempate.

Referencia al código. Los mecanismos se encuentran en las líneas 165 y 175 a 176.

4.7. Definición de la base de hechos

La base de hechos se construye dinámicamente a partir de dos fuentes: (i) el módulo de ontología en RDF y (ii) el sistema de lógica difusa, que interviene en la fase de desempate.

Traducción de la ontología (líneas 8 a 126)

El archivo ontologia_generada.ttl se carga en un grafo de rdflib y se recorre para convertir sus instancias en hechos. El proceso consta de los siguientes pasos:

1. **Funciones auxiliares.** n_local(uri) extrae el nombre local de un URI (líneas 15 a 16) y normalize(s) elimina espacios, tildes y mayúsculas para poder comparar textos (líneas 19 a 31).
2. **Conjuntos de clases y propiedades** (líneas 33 a 44). Se recorren las tripletas rdf:type rdfs:Class y rdf:type rdf:Property y se guardan los nombres locales en los conjuntos clase y propiedad, descartando los vocabularios estándar.
3. **Diccionario de personajes** (líneas 47 a 93). Para cada instancia de ex:Personaje (incluyendo las inferidas por el razonador de la ontología) se crea una entrada dict_personajes[nombre] y se evalúan todas las tripletas que la tienen como sujeto:
   - Literales enteros (valorPoder, valorAmenaza, valorPopularidad): se guardan como int.
   - foaf:name: se guarda el texto del nombre real.
   - Literales booleanos (usaIdentidadOculta, tieneCapa): se guarda el valor booleano.
   - rdf:type: la clase del objeto se guarda con valor True.
   - Cualquier otra propiedad presente: se guarda con valor True.
   - Toda clase o propiedad que el personaje no tenga se guarda con valor False (líneas 91 a 93), de manera que todos los personajes comparten exactamente las mismas claves.
4. **Banco de preguntas** (líneas 95 a 110). Lista de las 15 preguntas que puede formular el sistema.
5. **Asociación pregunta–elemento** (líneas 112 a 126). Cada clase o propiedad se asocia con la pregunta cuyo texto normalizado la contiene. Los elementos se procesan de mayor a menor longitud de nombre para evitar ambigüedades (por ejemplo, que Humano capture las preguntas de HumanoMutado o HumanoTecnologico). El resultado son los diccionarios clases_dic (10 elementos) y propiedades_dic (5 elementos).

Declaración de los hechos iniciales (líneas 427 a 442)

Una vez creado el motor (Akinator() y engine.reset()), se construyen y declaran con declare() los siguientes hechos:

- **Personaje:** 26 hechos, uno por cada personaje de la ontología (7 héroes de Marvel, 7 héroes de DC, 6 villanos de Marvel y 6 villanos de DC).
- **Clase:** 10 hechos (Superheroe, Supervillano, HeroeMarvel, HeroeDC, VillanoMarvel, VillanoDC, Humano, HumanoTecnologico, HumanoMutado, Alienigena), cada uno con su pregunta.
- **Propiedad:** 5 hechos (tienePoder, poseeArtefacto, esArchienemigoDe, usaIdentidadOculta, tieneCapa), cada uno con su pregunta.

Todos los hechos se mezclan aleatoriamente antes de declararse. Al final se declara EstadoJuego(fase="descarte") (línea 442) y se ejecuta el motor con engine.run(). La base de hechos inicial queda compuesta por **41 hechos** (26 + 10 + 5), más el hecho de control de la fase de juego.

Las clases Superpoder y Equipamiento, y las propiedades poseeArmaMitica, esEnemigoDe, valorPoder, valorAmenaza y valorPopularidad no generan preguntas ni hechos independientes: se usan como atributos de los hechos Personaje. La jerarquía de la ontología (por ejemplo HeroeMarvel como subclase de Superheroe, y este de Personaje) se aprovecha porque el razonador ya dejó cada personaje tipado con todas sus superclases, lo que permite que la respuesta a una pregunta general (¿es superhéroe?) descarte correctamente a los personajes de las subclases.

Integración con la lógica difusa

Cuando quedan exactamente dos candidatos, decision_final solicita al usuario los valores de poder, amenaza y popularidad, calcula con evaluar_perfil_difuso(poder, amenaza, popularidad) el impacto esperado y lo declara como hecho PerfilDifuso (línea 214). Este hecho, creado dinámicamente durante la ejecución y que por tanto no forma parte de los hechos iniciales, dispara la regla desempate, que calcula el impacto de cada candidato con los valores de la ontología y elige el más cercano al esperado. En caso de empate se conserva el primero de la lista. De este modo la ontología aporta el conocimiento estructurado y las reglas hacen el descarte, mientras que la lógica difusa introduce razonamiento bajo incertidumbre para el caso en que las respuestas de sí/no ya no distinguen a los candidatos.

4.8. Caso de prueba obligatorio: incorporación de un individuo desde la ontología

Este caso de prueba demuestra que el sistema experto se alimenta de la ontología y no de datos escritos en el código. Los personajes, sus clases y sus propiedades se leen del archivo ontologia_generada.ttl cada vez que se ejecuta el programa. Por eso, agregar un individuo nuevo en ese archivo basta para que el sistema lo reconozca como candidato, sin modificar el traductor ni el sistema experto.

Procedimiento: Después de ejecutar el módulo de ontología, se abre el archivo ontologia_generada.ttl y, debajo de cualquier personaje ya instanciado, se pegan las tripletas del nuevo individuo. En la prueba se agregó al villano Magneto (ex:Magneto), tipado como ex:VillanoMarvel, ex:Humano, ex:HumanoMutado, ex:Supervillano y ex:Personaje. Se le asignaron el artefacto ex:CascoAntiTelepatia, el poder ex:ControlMagnetico, tieneCapa en true, usaIdentidadOculta en false, valorPoder 67, valorAmenaza 5, valorPopularidad 40 y el nombre "Erik Lehnsherr". También se declararon ex:CascoAntiTelepatia como ex:Equipamiento y ex:ControlMagnetico como ex:Superpoder. Finalmente, se ejecuta de nuevo el sistema experto.

Por qué funciona sin modificar el código. El traductor recorre todas las tripletas de tipo ex:Personaje del grafo (línea 66), por lo que cualquier individuo nuevo con ese tipo se agrega automáticamente a dict_personajes. Sus clases y propiedades quedan en True, sus valores booleanos y enteros se copian de la ontología y lo que no tiene queda en False (líneas 91 a 93). Luego cada personaje se declara

como un hecho Personaje en el motor (líneas 429 a 430). Como las 30 reglas de descarte consultan atributos de forma general (por ejemplo, Personaje(VillanoMarvel=False)) y no nombran personajes específicos, Magneto queda cubierto sin reglas nuevas. Las preguntas del juego tampoco cambian, porque Magneto solo usa clases y propiedades que ya existen en la ontología.

Resultado esperado. Al iniciar, Magneto aparece en la lista de personajes y la base de hechos inicial pasa de 41 a 42 hechos. Si el usuario responde según las características de Magneto, las reglas de descarte eliminan a los demás candidatos y el sistema lo adivina. Si quedan dos candidatos, el desempate difuso usa los valores de Magneto en la ontología (poder 67, amenaza 5 y popularidad 40).

LINK DEL COLAB (entrar con el correo institucional):

https://colab.research.google.com/drive/1XpT-gJVNpYeaXikPRDgzqLap5WrhmeX-?usp=sharing

ENLACE VIDEO SUSTENTACIÓN:

https://youtu.be/a1A2eLDPrhI

5. Anexos

Fig. A: Diagrama de flujo del sistema experto

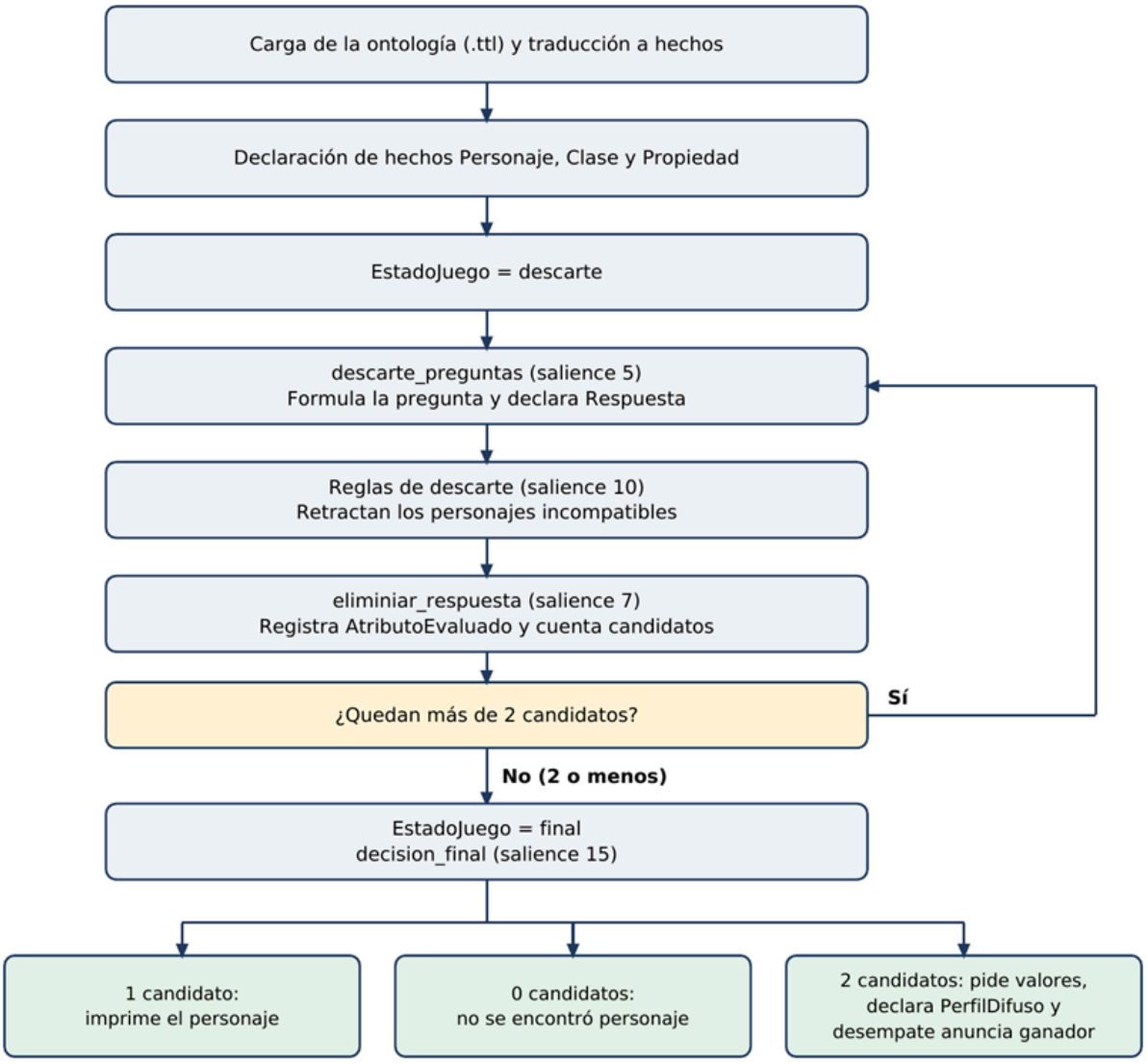


Fig. B: Diagrama de Clases UML

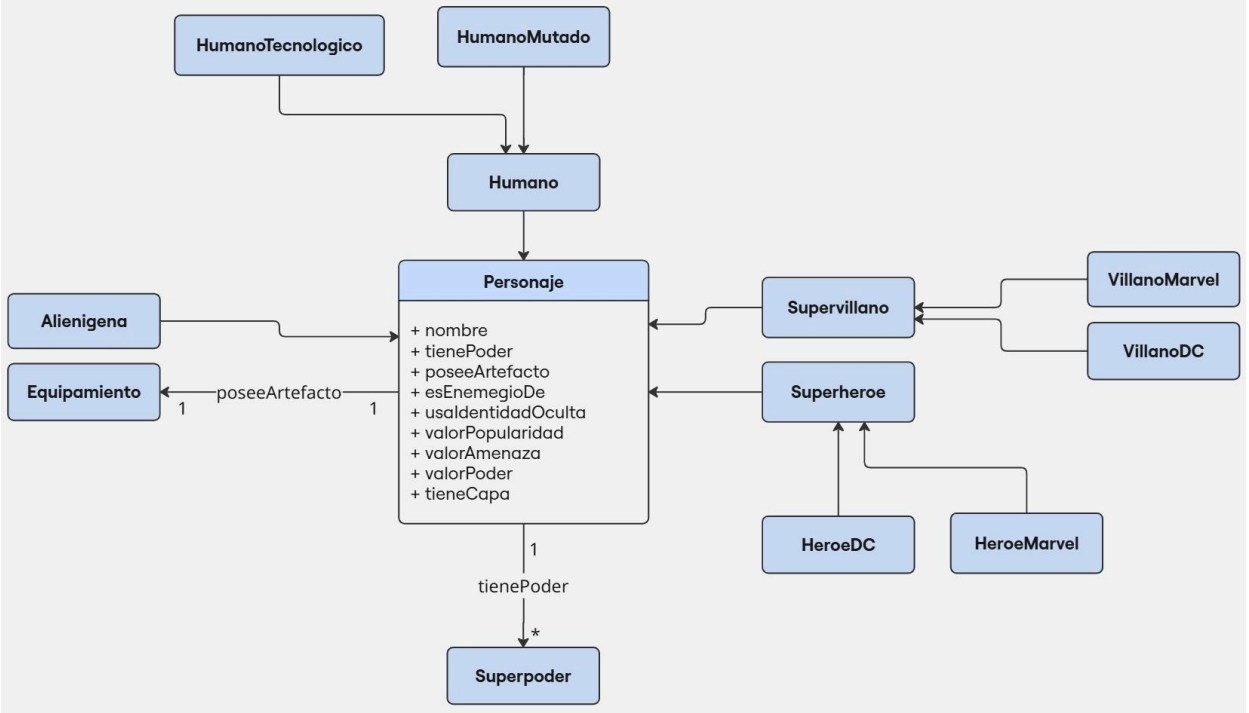


Fig. C: Grafo de Conocimiento Completo y Red Semántica

